

VELOSCOPE

Dossier d'analyse, conception et architecture technique



Stack : Python • FastAPI • PostgreSQL • pytest • ruff • Git

Source retenue : Vélib' Métropole / GBFS

Version de conception — septembre 2026

1. Présentation et objectifs

VeloScope est une API d'analyse de vélos en libre-service. Elle collecte périodiquement les données GBFS d'une ville, les historise dans PostgreSQL, puis produit des indicateurs temporels et des recommandations. Le point différenciant est l'historique : GBFS fournit l'état courant, VeloScope construit la mémoire du réseau.

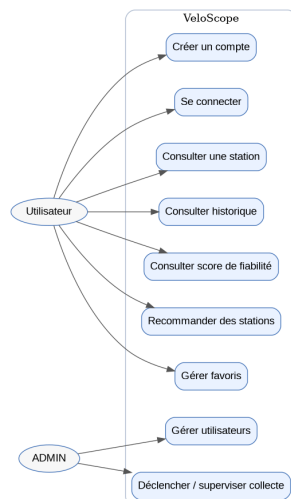
Objectifs métier

- connaître l'état actuel d'une station ;
- visualiser l'évolution sur 24 h, 7 j et 30 j ;
- mesurer la fréquence et la durée des pénuries/saturations ;
- attribuer un score de fiabilité explicable ;
- recommander plusieurs stations proches selon plusieurs critères ;
- préparer des extensions de prévision, heatmap et anomalies.

Vélib' Métropole est retenu car sa documentation officielle annonce des flux GBFS accessibles sans clé, en JSON, mis à jour chaque minute, avec notamment `station_information` et `station_status`. ■cite■turn0search0■

GBFS est un standard temps réel ; la documentation officielle précise qu'il n'est pas destiné à fournir un historique. L'historisation doit donc être réalisée par VeloScope. ■cite■turn0search4■turn0search10■

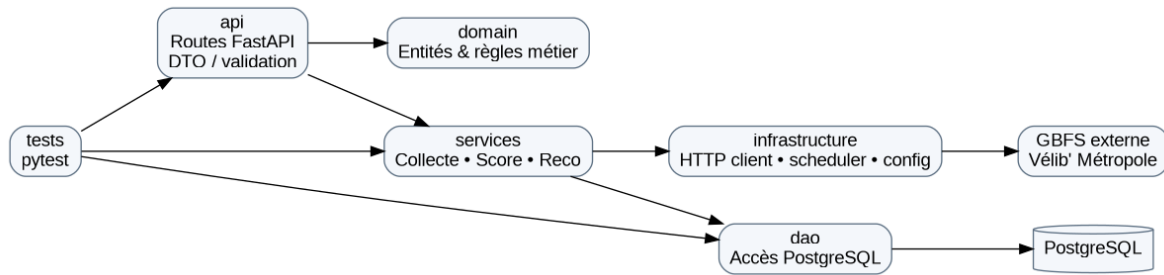
2. Fonctionnalités et cas d'utilisation



ID	Fonction	Description
F1	Utilisateurs	Compte, connexion, favoris, administration.
F2	Collecte	Interroger GBFS et conserver chaque relevé.
F3	Station	État, capacité, remplissage, historique.
F4	Fiabilité	Pourcentages vide/saturé, durées, score, évolution.
F5	Recommandation	Top stations selon distance + disponibilité + historique + e-bike + tendance.

ADMIN possède les droits de gestion des utilisateurs et de supervision de la collecte. L'utilisateur standard accède aux données, statistiques et favoris.

3. Architecture logicielle

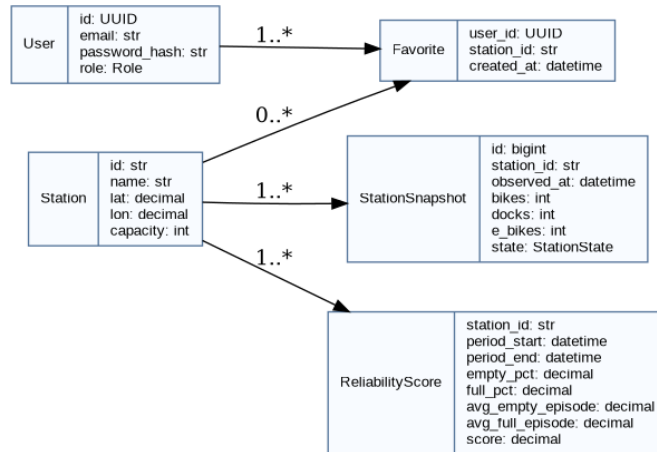


Organisation recommandée

```
veloscope/ app/api/ # routes FastAPI, auth, dépendances app/domain/ # entités, enums, règles app/services/ #  
collecte, fiabilité, recommandation app/dao/ # SQL / repositories app/infrastructure/ # client GBFS, scheduler,  
configuration app/schemas/ # DTO Pydantic tests/ # unitaires + intégration migrations/ # schéma PostgreSQL  
pyproject.toml
```

Le domaine ne dépend pas du fournisseur. Une interface GBFSProvider est implémentée par VelibProvider. Ainsi, un autre réseau GBFS peut être branché sans modifier les règles de fiabilité ou de recommandation.

4. Diagramme de classes



Entité	Responsabilité	Champs clés
User	Compte et rôle	id, email, password_hash, role
Station	Référentiel	id, name, lat, lon, capacity
StationSnapshot	Historique immuable	station_id, observed_at, bikes, docks, e_bikes, state
Favorite	Association utilisateur/station	user_id, station_id
ReliabilityScore	Résultat analytique	période, métriques, score

Enums : USER/ADMIN et OFFLINE/EMPTY/FULL/FUNCTIONAL. OFFLINE doit être distinct de EMPTY afin qu'une absence de données ne soit pas interprétée comme une station réellement vide.

5. Modèle de données PostgreSQL

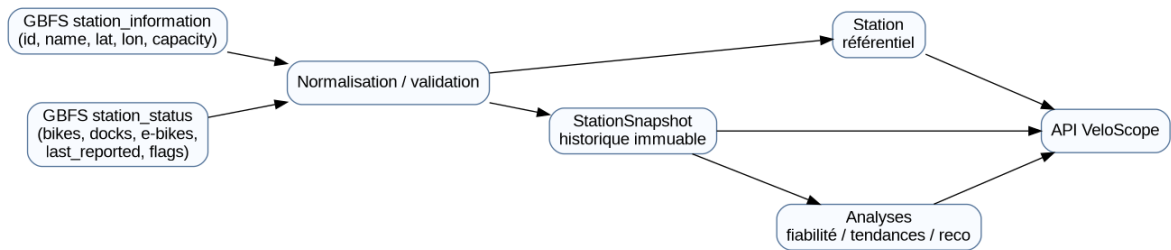


Table	Colonnes principales	Index / contraintes
users	id, email, password_hash, role, is_active	UNIQUE(email)
stations	id, name, lat, lon, capacity	PK(id)
station_snapshots	station_id, observed_at, bikes, docks, e_bikes, state, source_reported_at	FK + index (station_id, observed_at)
favorites	user_id, station_id, created_at	PK(user_id, station_id)
reliability_scores	station_id, période, métriques, score	index station + période
collection_runs	début, fin, statut, compteurs	audit/observabilité

Règle essentielle : **station_snapshots est append-only**. On met à jour le référentiel stations si nécessaire, mais on n’écrase jamais les relevés historiques. Pour les analyses, les durées sont calculées à partir des intervalles temporels réels entre observations.

6. API externe — Vélib' Métropole / GBFS

Flux documentés

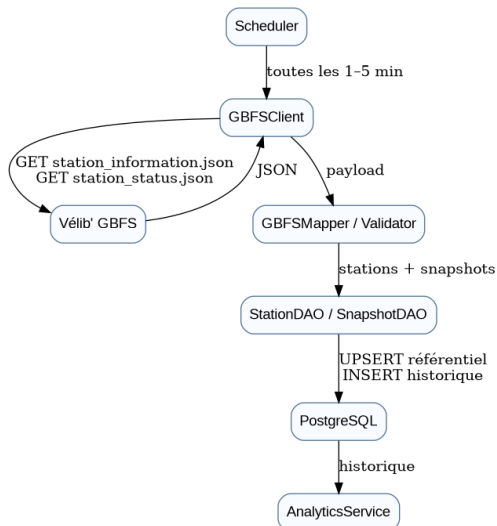
Feed	Usage
gbfs.json	Découverte des feeds et point d'entrée fournisseur.
system_information.json	Nom, identifiant, fuseau horaire.
station_information.json	ID, nom, latitude, longitude, capacité.
station_status.json	Vélos, places, e-bikes, disponibilité et horodatage.

Base documentée : https://velib-metropole-opendata.smovengo.cloud/opendata/Velib_Metropole/ . L'URL doit être externalisée en configuration et, idéalement, découverte via gbfs.json. Vélib' documente ses flux sous GBFS 1.0, tandis que la spécification officielle actuelle est v3.0 : le mapper doit donc isoler les différences de version. ■cite■turn0search0■turn0search2■

La documentation GBFS distingue explicitement les informations statiques et les données temps réel. Les flux near-real-time doivent rester proches du temps réel ; GBFS indique notamment une limite de fraîcheur de cinq minutes.

■cite■turn0search4■turn0search7■

7. Pipeline de collecte



Fréquence proposée : 1 à 5 minutes. Une minute est cohérente avec la cadence annoncée par Vélib'.

Erreur	Traitement
Timeout	Retry borné + backoff ; aucun snapshot fictif.
404	Signaler un problème de feed/configuration.
5xx	Retry ; conserver les dernières données valides.
JSON invalide	Rejeter le payload et journaliser.
Champ incohérent	Rejeter ou marquer la station selon règle métier.
Station absente	Ne pas supprimer le référentiel ; considérer la fraîcheur.

Chaque run est enregistré dans collection_runs afin de connaître le nombre de stations traitées, les erreurs et la durée.

8. Score de fiabilité

États : EMPTY si bikes = 0 ; FULL si docks = 0 ; FUNCTIONAL sinon ; OFFLINE si la station n'est pas opérationnelle ou si les données dépassent un seuil de fraîcheur.

EmptyPenalty = empty_pct

FullPenalty = full_pct

DurationPenalty = $0.5 \times \text{norm}(\text{avg_empty_duration}) + 0.5 \times \text{norm}(\text{avg_full_duration})$

Reliability = $100 \times (1 - 0.45 \times \text{EmptyPenalty} - 0.35 \times \text{FullPenalty} - 0.20 \times \text{DurationPenalty})$

Les pourcentages sont pondérés par durée et non par nombre de snapshots. Cela rend le calcul robuste lorsque deux collectes sont espacées de durées différentes.

Score	Interprétation
90–100	Très fiable
75–89	Fiable
50–74	Moyenne
0–49	Peu fiable

La fonction $\text{norm}(x)$ peut être définie par un seuil métier (par exemple 60 minutes = 1) ou par normalisation réseau. Cette décision doit être documentée et testée.

9. Recommandation de station



$$\text{RecommendationScore} = 0.35 \times \text{Availability} + 0.25 \times \text{Reliability} + 0.15 \times \text{EBike} + 0.15 \times \text{Trend} - 0.10 \times \text{DistancePenalty}$$

Critère	Définition
Availability	$\min(\text{bikes} / \text{seuil}, 1)$
Reliability	score de fiabilité / 100
EBike	$\text{e_bikes} / \max(1, \text{bikes})$
Trend	tendance récente normalisée sur [0,1]
DistancePenalty	distance / rayon max, bornée [0,1]

La réponse doit expliquer le classement : distance, vélos disponibles, e-bikes, fiabilité et tendance. Deux stations à distance identique peuvent ainsi être classées différemment.

10. Endpoints du backend FastAPI

Méthode	Endpoint	Fonction	Auth
POST	/api/v1/auth/register	Créer un compte	Public
POST	/api/v1/auth/login	Connexion/token	Public
GET	/api/v1/me	Profil	User
GET	/api/v1/stations	Recherche/liste	Public/User
GET	/api/v1/stations/{id}	État courant	Public/User
GET	/api/v1/stations/{id}/history	Historique 24h/7j/30j	Public/User
GET	/api/v1/stations/{id}/reliability	Score et métriques	Public/User
GET	/api/v1/stations/{id}/reliability/evolution	Évolution	Public/User
GET	/api/v1/recommendations	Top stations proches	Public/User
GET	/api/v1/favorites	Lister favoris	User
POST	/api/v1/favorites/{station_id}	Ajouter favori	User
DELETE	/api/v1/favorites/{station_id}	Supprimer favori	User
GET	/api/v1/admin/users	Lister utilisateurs	ADMIN
PATCH	/api/v1/admin/users/{id}	Modifier rôle/activation	ADMIN
GET	/api/v1/admin/collection/runs	Supervision collecte	ADMIN
POST	/api/v1/admin/collection/run	Collecte manuelle	ADMIN

11. Contrats JSON et paramètres

GET /api/v1/stations/{id}

```
{ "id": "123", "name": "Station Exemple", "location": {"lat": 48.85, "lon": 2.35}, "capacity": 35, "current": {"bikes": 12, "e_bikes": 5, "docks": 23, "state": "FUNCTIONAL", "observed_at": "2026-09-11T14:00:00+02:00"}, "fill_rate": 0.34 }
```

GET /api/v1/recommendations?lat=48.85&lon;=2.35&radius;_m=1200&limit;=3

```
{ "results": [{ "station_id": "123", "distance_m": 340, "score": 0.88, "bikes": 12, "e_bikes": 5, "reliability": 0.16, "reasons": ["proche", "fiable", "5 vélos électriques"] } ] }
```

Paramètres clés : lat, lon, radius_m, limit, electric_only ; pour l'historique : period=24h|7d|30d. Pagination ou agrégation recommandée pour les périodes longues.

12. Fonctionnalités optionnelles

ID	Extension	Conception
FO1	Prévision	Baseline par station + créneau + jour ; puis modèle ML si pertinent.
FO2	Heatmap	Agrégats géographiques des taux vide/saturé et fiabilité.
FO3	Anomalies	Feed muet, variation impossible, état figé, incohérences.
FO4	Assistant trajet	Fonctions de coût : rapide, fiable, e-bike, risque minimal.
FO5	Défi	Compteurs de trajets et badges ; nécessite une source d'usage réelle.

Pour FO1, une baseline statistique interprétable est préférable au départ. Pour FO3, les règles déterministes sont suffisantes pour une première version. Pour FO4, le moteur de routage peut être séparé de VeloScope ; VeloScope fournit alors les coûts liés aux stations.

13. Tests, qualité et sécurité

Niveau	Tests
Domaine	États, score de fiabilité, normalisation recommandation.
Mapper GBFS	JSON valide/invalid, champs absents, versions.
DAO	Insertion snapshots, historique, agrégations, contraintes.
API	Auth, rôles, paramètres invalides, erreurs.
Intégration	Feed → mapper → PostgreSQL ; retry ; panne fournisseur.
Non-régression	Historique conservé ; station absente ; changement de capacité.

Sécurité : hash Argon2/bcrypt, tokens non journalisés, contrôle ADMIN par dépendance FastAPI, validation des entrées.

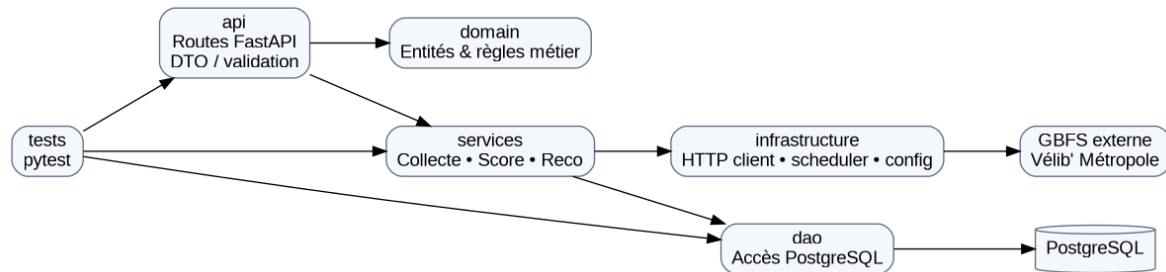
CI : **ruff check** . → **pytest** → **tests d'intégration**. Le pipeline Git doit bloquer un merge en cas d'échec.

14. Exploitation, volumétrie et observabilité

Métriques à suivre : durée des collectes, taux de succès, snapshots insérés, erreurs HTTP GBFS, âge des données et latence des recommandations.

La table `station_snapshots` peut devenir volumineuse. Une stratégie de rétention doit être décidée : conservation brute pour les périodes utiles, puis agrégations horaires/journalières. Si nécessaire, PostgreSQL peut être partitionné par mois sur `observed_at`.

Architecture d'extension



Le découpage permet d'ajouter une heatmap ou une prévision sans coupler ces fonctionnalités au client GBFS.

15. Plan de réalisation

Lot	Livrables	Sortie
L1	Socle FastAPI/PostgreSQL/pytest/ruff/Git	CI verte
L2	GBFSProvider + VelibProvider	Feeds consommés
L3	DAO + snapshots + scheduler	Historique multi-jours
L4	F3 station/historique	24h/7j/30j
L5	F4 fiabilité	Score testé
L6	F5 recommandation	Classement testé
L7	F1 auth/favoris/ADMIN	Permissions
L8	FO	Selon temps

Ordre recommandé : **données** → **historique** → **analytics** → **API** → **recommandation** → **administration** → **options**.

16. Risques et décisions d'architecture

Risque	Impact	Réponse
Changement GBFS	Élevé	Provider isolé + tests de contrat + découverte feed.
Panne externe	Élevé	Timeout, retry, observabilité, dernier état connu.
Volume historique	Élevé	Index, partitionnement, agrégations.
Données incohérentes	Élevé	Validation avant insertion.
Score contestable	Moyen	Formule explicable + tests + exemples réels.
OFFLINE = EMPTY	Élevé	États distincts + seuil de fraîcheur.

Décision clé : le score de fiabilité doit exploiter la continuité temporelle des snapshots. C'est ce qui permet de répondre à « combien de temps la station est-elle réellement restée vide ? » plutôt qu'à « combien de fois ai-je observé zéro vélo ? ».

17. Conclusion et sources

VeloScope répond au sujet en transformant un flux GBFS temps réel en un historique exploitable. La combinaison FastAPI + PostgreSQL + couche DAO + provider GBFS permet une architecture testable, extensible et indépendante du fournisseur. Le cœur métier repose sur la fiabilité temporelle et une recommandation multicritère explicable.

Sources techniques

- Vélib' Métropole — Open Data GBFS : flux, champs, URLs et fréquence de mise à jour. ■cite■turn0search0■
- GBFS — Documentation officielle et version de référence. ■cite■turn0search2■
- GBFS — Guide de démarrage et séparation statique/temps réel. ■cite■turn0search4■
- GBFS — FAQ sur la nature temps réel du standard et l'absence d'historique natif. ■cite■turn0search10■
- GBFS — Data Quality et recommandations de fraîcheur. ■cite■turn0search7■

Les URLs et versions des feeds doivent être revérifiées au moment de l'implémentation : un fournisseur peut faire évoluer ses endpoints ou sa version GBFS.